НЕГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ ЧАСТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ФИНАНСОВО-ПРОМЫШЛЕННЫЙ УНИВЕРСИТЕТ «СИНЕРГИЯ»

Кафедра Инновационных и цифровых технологий

ЛАБОРАТОРНАЯ РАБОТА № 2

Обучающийся Волков Илья Викторович

(ФИО) (подпись)

Преподаватель Сибирев Иван Валерьевич

(ФИО) (подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

Введение 3

1. Понятие развёртывания в DevOps 3

2. Этапы и артефакты процесса deploy 4

3. Стратегии развёртывания 5

4. Контейнеризация: Docker 6

5. Оркестрация: Kubernetes 7

6. Инфраструктура как код: Terraform 8

7. Бессерверные вычисления: AWS Lambda 8

8. GitOps и Argo CD 9

9. Мониторинг: Prometheus и Datadog 10

Заключение 10

Список использованных источников 11

ВВЕДЕНИЕ

Современная разработка программного обеспечения не ограничивается написанием кода и его тестированием. После того как код написан, собран и проверен, наступает не менее важный этап — введение программного продукта в эксплуатацию, то есть его развёртывание (deploy) в целевой среде и последующее сопровождение. Именно на этом этапе продукт становится доступным конечному пользователю, а разработчики и инженеры эксплуатации получают возможность оценить его реальное поведение под нагрузкой.

В рамках методологии DevOps процесс развёртывания перестал быть разовым событием. Он превратился в непрерывный автоматизированный процесс, тесно связанный с предшествующими этапами CI/CD и последующими этапами эксплуатации и мониторинга. Современный deploy — это сложная инженерная дисциплина, включающая контейнеризацию приложений, оркестрацию вычислительных ресурсов, управление инфраструктурой как кодом, реализацию продвинутых стратегий выката и постоянный мониторинг состояния систем.

Цель данной работы — рассмотреть основные технологии и практики, применяемые на этапе развёртывания в современной DevOps-экосистеме. В работе будут последовательно изучены понятие и этапы deploy, применяемые стратегии выката, ключевые инструменты (Docker, Kubernetes, Terraform, AWS Lambda, Argo CD), а также системы мониторинга, замыкающие цикл эксплуатации (Prometheus, Datadog).

1. ПОНЯТИЕ РАЗВЁРТЫВАНИЯ В DEVOPS

Развёртывание (deployment) — это процесс переноса программного продукта из среды разработки или тестирования в среду, где им будут пользоваться конечные потребители. В широком смысле deploy включает в себя не только копирование кода на серверы, но и подготовку инфраструктуры,

настройку сетевого взаимодействия, применение конфигураций, миграцию данных и обеспечение доступности сервиса.

В классической модели DevOps, представляемой в виде бесконечной петли, этап Deploy располагается на границе между разработкой и эксплуатацией. Он завершает цикл CD (Continuous Delivery) и открывает цикл работы с системой в продакшене — Operate и Monitor. Именно качество реализации этапа развёртывания во многом определяет, насколько быстро и безопасно команда может выпускать новые версии.

Современные требования к процессу deploy сформировались под влиянием нескольких факторов: роста масштаба и сложности распределённых систем, высоких ожиданий пользователей относительно доступности сервисов, необходимости быстро реагировать на изменения рынка. В результате к процессу развёртывания предъявляются следующие основные требования:

- Автоматизация — исключение ручных операций, подверженных ошибкам;
- Воспроизводимость — одна и та же версия должна разворачиваться одинаково в любой среде;
- Идемпотентность — повторный запуск процесса не должен приводить к побочным эффектам;
- Атомарность — развёртывание либо полностью применяется, либо полностью откатывается;
- Обратимость — возможность быстрого отката к предыдущей версии при возникновении проблем;
- Минимизация простоя — пользователи не должны испытывать неудобств во время выкатки;
- Наблюдаемость — все действия должны логироваться и отслеживаться.

Выполнение этих требований обеспечивается за счёт применения специализированных инструментов и практик, которые и составляют предмет настоящей работы.

2. ЭТАПЫ И АРТЕФАКТЫ ПРОЦЕССА DEPLOY

Процесс развёртывания современного приложения представляет собой последовательность взаимосвязанных этапов, каждый из которых производит определённый результат — артефакт. Типовая схема выглядит следующим образом:

1. Подготовка артефакта. Исходный код после успешного прохождения CI упаковывается в развёртываемую форму. Это может быть JAR-файл для Java-приложения, tar-архив для Node.js, но чаще всего — Docker-образ, содержащий приложение вместе со всеми его зависимостями.

2. Публикация в реестр. Готовый артефакт помещается в специализированное хранилище: Docker Registry для контейнерных образов, Nexus или Artifactory для традиционных пакетов. Каждый артефакт получает уникальный идентификатор (тег или версию).

3. Подготовка инфраструктуры. Целевая среда должна быть готова принять новую версию: настроены серверы, кластеры, сетевые правила, балансировщики, хранилища. Современный подход — описывать эту инфраструктуру декларативно в коде (Infrastructure as Code).

4. Применение конфигурации. Приложению передаются параметры, специфичные для среды: адреса баз данных, учётные данные, флаги функциональности. Используются инструменты управления секретами (HashiCorp Vault, Kubernetes Secrets).

5. Миграция данных. Если новая версия требует изменений схемы базы данных, они применяются с помощью инструментов миграции (Liquibase, Flyway). Особое внимание уделяется обратной совместимости.

6. Выкат приложения. Непосредственное развёртывание новой версии согласно выбранной стратегии (rolling, blue-green, canary и др.). На этом этапе происходит постепенная или одномоментная замена работающих экземпляров.

7. Проверка работоспособности. После развёртывания выполняются smoke-тесты и проверки health-эндпоинтов, подтверждающие корректность работы новой версии.

8. Мониторинг и наблюдение. В течение некоторого времени после выката команда эксплуатации внимательно следит за метриками, готовая выполнить откат при обнаружении аномалий.

3. СТРАТЕГИИ РАЗВЁРТЫВАНИЯ

Под стратегией развёртывания понимается способ замены работающей версии приложения на новую. Выбор стратегии определяется требованиями к доступности сервиса, толерантностью к рискам и доступными ресурсами инфраструктуры. Рассмотрим наиболее распространённые стратегии.

Recreate (пересоздание) — самая простая стратегия: все экземпляры старой версии останавливаются, затем запускаются экземпляры новой. Применяется в случаях, когда допустим короткий простой и когда старая и новая версии не могут сосуществовать (например, из-за несовместимых изменений схемы данных).

Rolling Update (поэтапное обновление) — постепенная замена экземпляров: новые запускаются, а старые останавливаются порциями. В любой момент часть экземпляров работает со старой версией, часть — с новой. Это стратегия по умолчанию в Kubernetes.

Blue-Green Deployment — параллельно поддерживаются две полные среды: работающая «синяя» (blue) и подготовленная «зелёная» (green) с новой версией. После проверки готовности трафик одномоментно переключается с

blue на green. Откат осуществляется за секунды — достаточно вернуть трафик на blue.

Canary Deployment (канареечное развёртывание) — новая версия сначала разворачивается для небольшой доли пользователей (5–10%), а затем при успешных результатах её доля постепенно увеличивается. Название происходит от практики шахтёров использовать канареек для обнаружения опасных газов.

A/B Testing — технически похожа на canary, но применяется не для минимизации рисков релиза, а для проверки бизнес-гипотез: разные сегменты пользователей получают разные варианты функциональности, после чего анализируется их поведение.

Сравнительная характеристика стратегий представлена в таблице ниже:

Стратегия	Принцип	Преимущества	Недостатки
Recreate	Остановка старой версии, запуск новой	Простота реализации	Простой сервиса во время развёртывания
Rolling Update	Постепенная замена экземпляров старой версии новыми	Нет простоя, экономия ресурсов	Сосуществование версий, сложность отката
Blue-Green	Две параллельные среды, переключение трафика	Мгновенный откат, нет простоя	Двойной расход инфраструктурных ресурсов
Canary	Новая версия для малой доли пользователей с постепенным расширением	Снижение рисков, сбор метрик	Сложность настройки маршрутизации трафика
A/B Testing	Разные версии для разных сегментов пользователей	Проверка гипотез, анализ поведения	Не предназначена для релизов как таковых

4. КОНТЕЙНЕРИЗАЦИЯ: DOCKER

Docker — платформа контейнеризации, появившаяся в 2013 году и радикально изменившая подход к развёртыванию приложений. Контейнер представляет собой изолированную среду, включающую приложение вместе со всеми необходимыми для его работы зависимостями: системными библиотеками, интерпретаторами, утилитами и конфигурационными файлами. В отличие от виртуальных машин контейнеры используют общее ядро операционной системы хоста, что делает их существенно более лёгкими и быстрыми.

Ключевым элементом Docker-экосистемы является образ (image) — неизменяемый шаблон, описываемый декларативно в файле Dockerfile. Образы строятся из слоёв, что обеспечивает их эффективное хранение и передачу: при обновлении приложения по сети передаются только изменившиеся слои.

Пример простейшего Dockerfile для Node.js-приложения:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Контейнеризация решает классическую проблему «у меня работало»: образ, собранный на машине разработчика, гарантированно будет работать одинаково в тестовой и производственной средах. Это обеспечивает воспроизводимость развёртывания — одно из ключевых требований современного deploy.

Среди преимуществ Docker можно выделить переносимость между различными платформами, изоляцию процессов, лёгкость масштабирования за счёт запуска дополнительных контейнеров, а также экономию ресурсов по сравнению с виртуальными машинами. Именно контейнеризация стала технологической основой, на которой построены современные системы оркестрации.

5. ОРКЕСТРАЦИЯ: KUBERNETES

Kubernetes (часто сокращается до K8s) — система управления контейнеризированными приложениями, разработанная компанией Google и переданная в 2014 году в фонд Cloud Native Computing Foundation. Kubernetes решает задачу оркестрации: запуска, масштабирования, балансировки и восстановления контейнеров в кластере, состоящем из множества физических или виртуальных машин.

В основе Kubernetes лежит декларативная модель: администратор описывает желаемое состояние системы в виде YAML-манифестов, а платформа самостоятельно приводит фактическое состояние кластера в соответствие с описанным. Если какой-либо компонент выходит из строя, Kubernetes автоматически запускает его заново; если нагрузка возрастает — запускает дополнительные экземпляры.

Основные сущности Kubernetes, связанные с развёртыванием:

- Pod — минимальная единица развёртывания, содержащая один или несколько контейнеров;
- Deployment — объект, управляющий набором подов и реализующий стратегии rolling update и recreate;
- Service — механизм, обеспечивающий стабильный сетевой доступ к подам через балансировщик нагрузки;
- Ingress — правила маршрутизации внешнего HTTP/HTTPS-трафика к сервисам;

- ConfigMap и Secret — объекты для хранения конфигураций и секретных данных;
- Namespace — логическое разделение ресурсов внутри одного кластера.

Пример манифеста Deployment, разворачивающего три реплики веб-приложения со стратегией rolling update:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: registry.example.com/web-app:v1.2.3
        ports:
        - containerPort: 3000
        resources:
          limits:
```

```
            memory: "256Mi"
            cpu: "500m"
```

Kubernetes стал де-факто стандартом оркестрации контейнеров и поддерживается всеми ведущими облачными провайдерами (Amazon EKS, Google GKE, Azure AKS). Вокруг него сформировалась обширная экосистема сопутствующих инструментов: Helm для шаблонизации манифестов, Istio и Linkerd для сервис-мешей, различные операторы для управления базами данных и другими stateful-системами.

6. ИНФРАСТРУКТУРА КАК КОД: TERRAFORM

Terraform — инструмент управления инфраструктурой как кодом (Infrastructure as Code, IaC), разработанный компанией HashiCorp в 2014 году. В отличие от инструментов конфигурации серверов (Ansible, Chef, Puppet), Terraform работает на более высоком уровне — он управляет созданием и изменением самих инфраструктурных ресурсов: виртуальных машин, сетей, балансировщиков, кластеров Kubernetes, облачных баз данных.

Инфраструктура описывается на декларативном языке HCL (HashiCorp Configuration Language). Terraform хранит состояние управляемой инфраструктуры (state file) и при каждом запуске сравнивает желаемое состояние из конфигурации с фактическим, вычисляя минимальный набор изменений для их синхронизации.

Пример конфигурации, создающей виртуальный сервер в AWS:

```
provider "aws" {
  region = "eu-central-1"
}

resource "aws_instance" "web_server" {
  ami           = "ami-0abcd1234efgh5678"
  instance_type = "t3.medium"
```

```
    tags = {
      Name        = "web-server-prod"
      Environment = "production"
    }
  }
```

Ключевые преимущества IaC-подхода: версионирование инфраструктуры в Git, возможность code review изменений, воспроизводимость сред (dev, staging, production создаются из одного кода с разными параметрами), автоматизация и снижение зависимости от конкретных специалистов. Terraform поддерживает сотни провайдеров — от AWS, Google Cloud и Azure до сервисов мониторинга, DNS-провайдеров и систем управления пользователями.

7. БЕССЕРВЕРНЫЕ ВЫЧИСЛЕНИЯ: AWS LAMBDA

AWS Lambda — сервис бессерверных вычислений (Function as a Service, FaaS) от Amazon Web Services, запущенный в 2014 году. Парадигма serverless не означает отсутствие серверов как таковых — она означает, что разработчик избавлен от необходимости заботиться об их предоставлении, настройке и масштабировании. Вместо этого он просто загружает код функции, а облачная платформа берёт на себя всю инфраструктурную часть.

Модель работы AWS Lambda основана на событиях: функция запускается в ответ на определённые триггеры — HTTP-запрос через API Gateway, появление файла в хранилище S3, сообщение в очереди SQS, изменение записи в базе данных DynamoDB и многие другие. Оплата производится только за фактическое время выполнения функций с точностью до миллисекунды, что делает эту модель чрезвычайно экономичной для нагрузок с нерегулярным характером.

Характерные особенности serverless-развёртывания:

- Автоматическое масштабирование от нуля до тысяч параллельных вызовов без настройки;
- Отсутствие необходимости управлять серверами, обновлять ОС и библиотеки;
- Встроенная отказоустойчивость на уровне платформы;
- Ограничения на время выполнения (до 15 минут в Lambda) и объём памяти;
- Проблема «холодного старта» — задержка при первом вызове неактивной функции.

Serverless-подход хорошо подходит для обработки веб-запросов с переменной нагрузкой, фоновых задач, интеграций между системами, обработки потоков данных. Однако для приложений с постоянной высокой нагрузкой традиционные подходы с контейнерами в Kubernetes могут оказаться более экономичными. Аналоги AWS Lambda существуют у всех крупных облачных провайдеров: Google Cloud Functions, Azure Functions, Yandex Cloud Functions.

8. GITOPS И ARGO CD

GitOps — современный подход к развёртыванию и эксплуатации инфраструктуры, сформулированный компанией Weaveworks в 2017 году. Его основная идея заключается в том, что Git-репозиторий становится единственным источником истины (single source of truth) о желаемом состоянии системы, а все изменения в продакшене производятся исключительно через Git-коммиты.

Четыре основных принципа GitOps:

- Декларативность — вся система описывается декларативно, а не через последовательность команд;
- Версионирование в Git — описание системы хранится в Git-репозитории с полной историей изменений;

- Автоматическое применение — одобренные изменения в Git автоматически применяются к системе;
- Непрерывная синхронизация — специальный агент постоянно сверяет фактическое состояние с описанным в Git и устраняет расхождения.

Argo CD — наиболее популярная реализация GitOps-подхода для Kubernetes. Инструмент был создан компанией Intuit в 2018 году и является выпускным проектом CNCF. Argo CD устанавливается в Kubernetes-кластер в виде набора контроллеров и предоставляет веб-интерфейс для визуализации состояния развёрнутых приложений.

Принципиальным отличием Argo CD от традиционных CI/CD-систем является pull-модель развёртывания. Классические инструменты (Jenkins, GitLab CI) используют push-модель: они подключаются к кластеру извне и отправляют туда изменения. Argo CD работает изнутри: его агент периодически опрашивает Git-репозиторий и самостоятельно применяет обнаруженные изменения. Такая архитектура обеспечивает:

- Повышенную безопасность — учётные данные кластера не покидают его пределов;
- Автоматическое обнаружение и исправление отклонений (drift detection);
- Полную прозрачность и аудируемость всех изменений через историю Git;
- Простоту отката — достаточно revert-коммита в Git.

На практике Argo CD часто используется в связке с классическими CI-системами: GitLab CI или GitHub Actions отвечают за сборку и публикацию Docker-образов, после чего обновляют манифесты Kubernetes в отдельном Git-репозитории. Argo CD обнаруживает изменения и применяет их к кластеру. Такое разделение ответственности между CI и CD считается современной best-practice при работе с Kubernetes. Родственный проект Argo Rollouts расширяет

возможности Argo CD поддержкой продвинутых стратегий развёртывания — canary и blue-green.

9. МОНИТОРИНГ: PROMETHEUS И DATADOG

Развёртывание приложения не заканчивается его запуском в продакшене — оно продолжается постоянным наблюдением за его работой. Мониторинг замыкает цикл DevOps, передавая обратную связь от работающей системы разработчикам и инженерам эксплуатации. В случае обнаружения проблем именно мониторинг становится триггером для отката или выпуска исправляющего релиза.

Prometheus — система мониторинга с открытым исходным кодом, разработанная компанией SoundCloud в 2012 году и ставшая вторым (после Kubernetes) выпускным проектом CNCF. Prometheus собирает метрики от отслеживаемых систем по pull-модели: периодически опрашивает специальные HTTP-эндпоинты, на которых приложения публикуют свои показатели. Собранные данные хранятся в собственной time-series базе данных и могут быть извлечены с помощью специализированного языка запросов PromQL.

Ключевые особенности Prometheus:

- Многомерная модель данных — метрики характеризуются не только именем и значением, но и набором меток (labels);
- Мощный язык запросов PromQL для агрегации и анализа данных;
- Встроенная система оповещений через компонент Alertmanager;
- Автоматическое обнаружение целей мониторинга (service discovery), особенно тесно интегрированное с Kubernetes;
- Широкая экосистема экспортёров (exporters) для мониторинга баз данных, сетевого оборудования, аппаратных показателей.

Datadog — коммерческая облачная платформа мониторинга и наблюдаемости (observability). В отличие от Prometheus, который требует

самостоятельного развёртывания и сопровождения, Datadog предоставляется как SaaS-решение: клиент устанавливает агентов на свои системы, а платформа берёт на себя сбор, хранение, визуализацию и оповещения. Datadog объединяет в едином интерфейсе три классические составляющие наблюдаемости: метрики (metrics), логи (logs) и трассировки (traces), что позволяет быстро анализировать инциденты, переходя между различными представлениями одних и тех же данных.

Выбор между Prometheus и Datadog зависит от потребностей организации. Prometheus бесплатен, обеспечивает полный контроль над данными и хорошо интегрируется с Kubernetes, но требует собственной инфраструктуры и экспертизы для сопровождения. Datadog предоставляет более богатую функциональность «из коробки», развитую визуализацию и поддержку, но требует оплаты по модели подписки, которая при большом объёме данных может быть существенной. В крупных организациях часто применяется гибридный подход: Prometheus для сбора детальных технических метрик и Datadog для бизнес-дашбордов и корреляции данных из различных источников.

ЗАКЛЮЧЕНИЕ

В ходе выполнения настоящей работы были рассмотрены ключевые аспекты этапа развёртывания и последующего введения программного продукта в эксплуатацию в рамках методологии DevOps. Установлено, что современный процесс deploy представляет собой сложную инженерную дисциплину, требующую применения специализированных инструментов и продуманных стратегий.

Были изучены основные технологические составляющие современного развёртывания: контейнеризация на базе Docker, обеспечивающая воспроизводимость среды выполнения; оркестрация Kubernetes, реализующая автоматическое управление жизненным циклом контейнеров; инфраструктура как код с применением Terraform, позволяющая описывать всю

вычислительную среду декларативно; бессерверные вычисления AWS Lambda, дающие возможность полностью абстрагироваться от управления серверами; GitOps-подход с реализацией в Argo CD, превращающий Git-репозиторий в единый источник истины о состоянии системы.

Отдельно рассмотрены системы мониторинга Prometheus и Datadog, играющие ключевую роль в замыкании цикла DevOps: они обеспечивают обратную связь от работающей системы, позволяя команде оперативно реагировать на инциденты и планировать улучшения.

Совокупное применение рассмотренных инструментов и практик позволяет современным командам разработки достигать того, что ещё десять лет назад казалось невозможным: десятки и сотни развёртываний в сутки, нулевое время простоя при обновлениях, автоматическое масштабирование в соответствии с нагрузкой и быстрое восстановление после инцидентов. Именно эти возможности лежат в основе конкурентоспособности современных технологических компаний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ким Дж., Дебуа П., Уиллис Дж., Хамбл Дж. Руководство по DevOps. — М.: Манн, Иванов и Фербер, 2018. — 416 с.
2. Хамбл Дж., Фарли Д. Непрерывное развёртывание ПО: автоматизация процессов сборки, тестирования и внедрения новых версий программ. — М.: Вильямс, 2011. — 432 с.
3. Бёрнс Б., Хайтауэр К., Бергер Д. Kubernetes в действии. — М.: ДМК Пресс, 2019. — 528 с.
4. Моуэт Э. Использование Docker. — М.: ДМК Пресс, 2017. — 354 с.
5. Брикман Е. Terraform: инфраструктура на уровне кода. — СПб.: Питер, 2020. — 368 с.
6. Официальная документация Docker [Электронный ресурс]. — URL: https://docs.docker.com (дата обращения: 24.04.2026).

7. Официальная документация Kubernetes [Электронный ресурс]. — URL: https://kubernetes.io/docs/ (дата обращения: 24.04.2026).
8. Документация Terraform [Электронный ресурс]. — URL: https://developer.hashicorp.com/terraform (дата обращения: 24.04.2026).
9. Документация AWS Lambda [Электронный ресурс]. — URL: https://docs.aws.amazon.com/lambda/ (дата обращения: 24.04.2026).
10. Документация Argo CD [Электронный ресурс]. — URL: https://argo-cd.readthedocs.io (дата обращения: 24.04.2026).